

HPC Integration of Agents

Amit Gupta

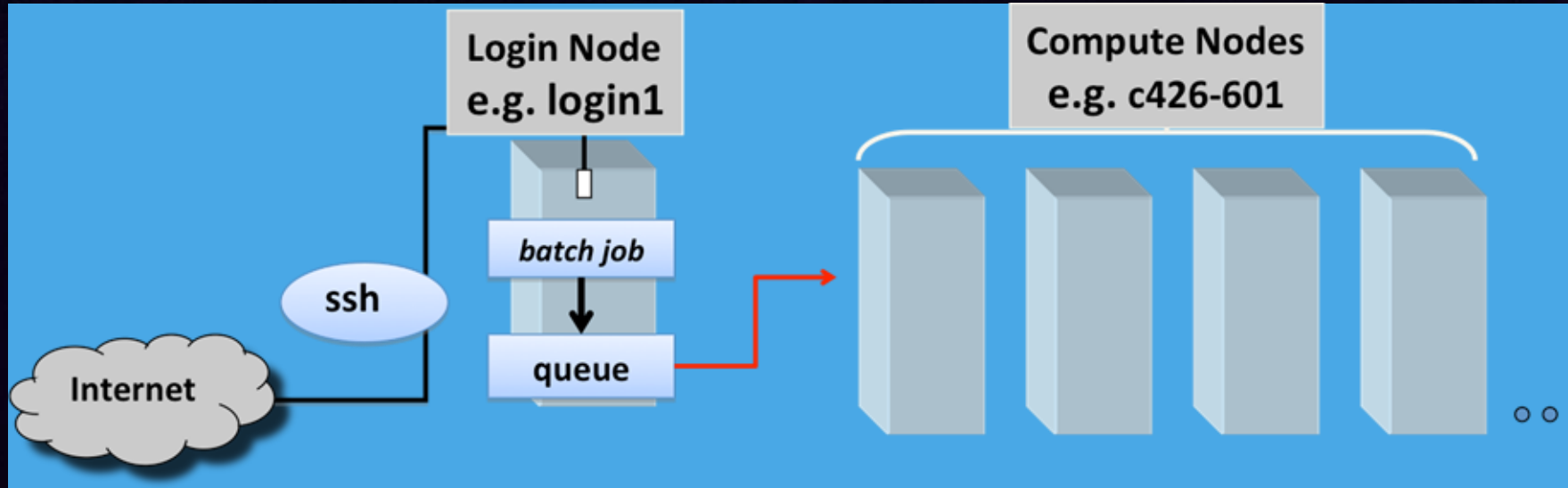
March 24, 2026

Scalable Computational Intelligence Group (SCI)

Texas Advanced Computing Center (TACC)



Login vs Compute Nodes



Login Nodes

- Shared by multiple users
- Not meant for CPU or memory intensive jobs
- Meant for managing file and compiling code

Compute Nodes

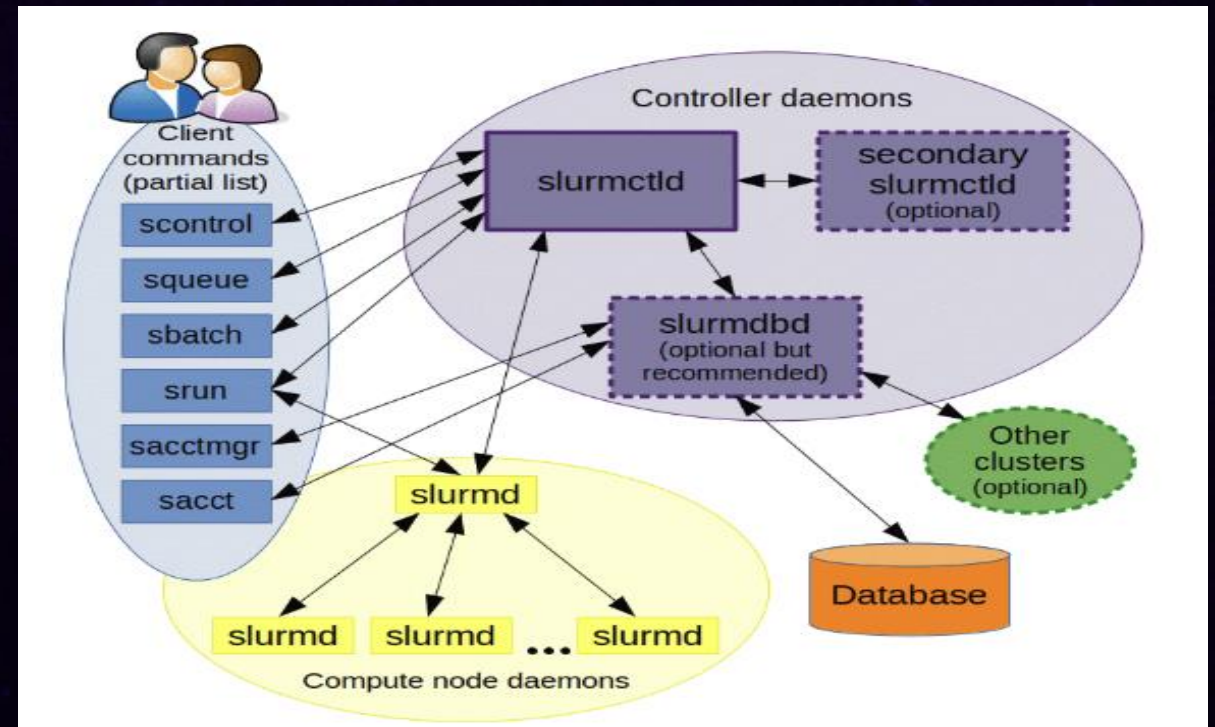
- Where research computation occurs
- Training machine learning models, running simulations, etc.

SLURM

- Workload manager commonly used in HPC machines
- Allocates and controls access to compute nodes to users for some time duration
- Provides mechanisms to
 - Start
 - Execute
 - Monitor
- Manages contention by maintaining a job queue for selected resources
 - Partitions (group of certain type of compute nodes)
- Extensible
 - Provides a plugin architecture to extend functionality
 - Authentication
 - Accounting

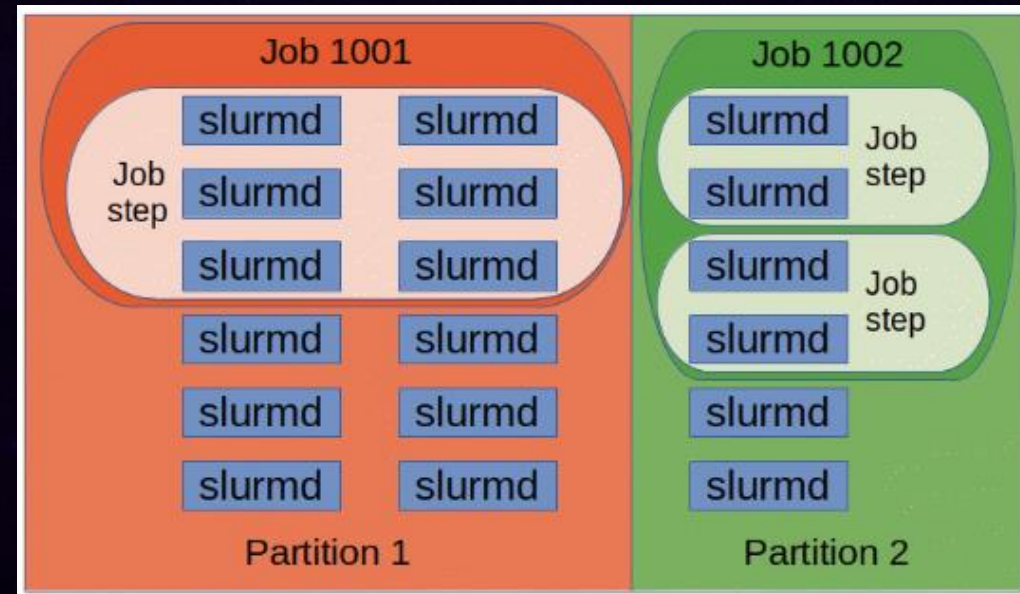
Architecture

- Command line tools
 - scontrol
 - squeue
 - sbatch
 - srun
 - sinfo
- Daemons
 - slurmctld (Head node of the cluster)
 - slurmctld (2) (Secondary fail over)
 - slurmdbd (Database daemon)
 - slurmd (Compute node daemon)



Slurm Entities

- Nodes
 - Unit of resource allocation
- Partitions
 - Groups of nodes
 - Hardware features
 - Special purposes
- Jobs
 - Unit of scheduled work
- Job steps
 - Tasks within jobs
 - Often parallel
 - Use all or part of allocated resources
- Job Arrays
 - Run same code with different data



queue

queue:

view information about jobs located in the Slurm scheduling queue

Usage: queue [OPTIONS]

-A, --account=account(s)	comma separated list of account to view
-i, --iterate=seconds	specify an iteration period
-j, --job=job(s)	comma separated list of jobs IDs
-S, --sort=fields	comma separated list of fields to sort on
--start	print expected start times of pending jobs
-t, --states=states	comma separated list of states to view
-u, --user=user_name(s)	comma separated list of users to view
--name=job_name(s)	comma separated list of job names to view

sinfo

sinfo:

View information about Slurm nodes and partitions.

Usage: sinfo [OPTIONS]

-a, --all	show all partitions
-i, --iterate=seconds	specify an iteration period
-n, --nodes=NODES	report on specific node(s)
-p, --partition=PARTITION	report on specific partition
-s, --summarize	report state summary only
-t, --states=node_state	specify the what states of nodes to view
-R, --list-reasons	list reason nodes are down or drained
-r, --responding	report only responding nodes
-T, --reservation	show only reservation information

sacct

sacct:

View accounting information for all running jobs of this account.

sacct [<OPTION>]

Valid <OPTION> values are:

-a, --allusers:

Display jobs for all users. By default, only the current user's jobs are displayed. If ran by user root this is the default.

-s, --state=<state_list>

Selects jobs based on their state --array:
Expand job arrays. Display array tasks on separate lines instead of consolidating them to a single line.

-b, --brief:

Equivalent to '--format=jobstep,state,error'.

-B, --batch-script:

Print batch script of job.

NOTE: AccountingStoreFlags=job_script is required for this

NOTE: Requesting specific job(s) with '-j' is required for this.

scontrol

scontrol:

view or modify Slurm configuration and state.

Usage: scontrol [OPTIONS] [COMMAND]

- | | |
|---------------|--|
| -a, --all | Equivalent to "all" command |
| -d, --details | Equivalent to "details" command |
| -u, --uid | Update job as user "uid" instead of the invoking user. |

Commands

- | | |
|--------|------------------------------------|
| show | show entity (ex. "reservations") |
| resume | allow a job to be scheduled |
| hold | prevent a job from being scheduled |

srun

srun:

Run an MPI job.

Usage: srun [OPTIONS(0)... [executable(0) [args(0)...]]] [: [OPTIONS(N)...] executable(N) [args(N)...]

Parallel run options:

-A, --account=name	charge job to specified account
-b, --begin=time	defer job until HH:MM MM/DD/YY
-c, --cpus-per-task=ncpus	number of cpus required per task
-d, --dependency=type:jobid[:time]	defer job until condition on jobid is satisfied
-D, --chdir=path	change remote current working directory
-n, --ntasks=ntasks	number of tasks to run
--ntasks-per-node=n	number of tasks to invoke on each node
-N, --nodes=N	number of nodes on which to run (N = min[-max])

sbatch

sbatch:

Submit a batch job script.

Usage: sbatch [OPTIONS] [Job Script]

-n, --ntasks=ntasks	number of tasks to run
-N, --nodes=N	number of nodes on which to run (N = min[-max])
-p, --partition=partition	partition requested
-t, --time=minutes	time limit
-o, --output=out	file for batch script's standard output



scancel

scancel:

Cancel a running job/job array.

Usage: Usage: scancel [OPTIONS] [job_id[_array_id][.step_id]]

- | | |
|-------------------------------|---|
| -A, --account=account | act only on jobs charging this account |
| -i, --interactive | require response from user for each job |
| -n, --name=job_name | act only on jobs with this name |
| -p, --partition=partition | act only on jobs in this partition |
| -R, --reservation=reservation | act only on jobs with this reservation |
| --sibling=cluster_name | remove an active sibling job from a federated job |
| -s, --signal=name integer | signal to send to job, default is SIGKILL |
| -t, --state=states | act only on jobs in this state. Valid job states are PENDING, RUNNING and SUSPENDED |
| -u, --user=user_name | act only on jobs of this user |
| -w, --odelist | act only on jobs on these nodes |

Others

sstat

Display status information of running job.

sprio

View factors affecting jobs scheduling priority (with weights)

strigger

View or set trigger information (trigger events)

sattach

Attach to stdout of a specific job

Job submission script example

```
#!/bin/bash
#SBATCH -J mps_gpu_job # Job name
#SBATCH -o mps_job.%j.out # Output file (%j = job ID)
#SBATCH -t 01:00:00 # Wall time (1 hour)
#SBATCH -N 2 # Number of nodes
#SBATCH -n 8 # Total tasks (4 per node)
#SBATCH -p gh # GPU partition (modify as needed)
#SBATCH -A your_project # Project allocation

# 1. Configure environment
export CUDA_MPS_PIPE_DIRECTORY=/tmp/nvidia-mps
export CUDA_MPS_LOG_DIRECTORY=/tmp/nvidia-log

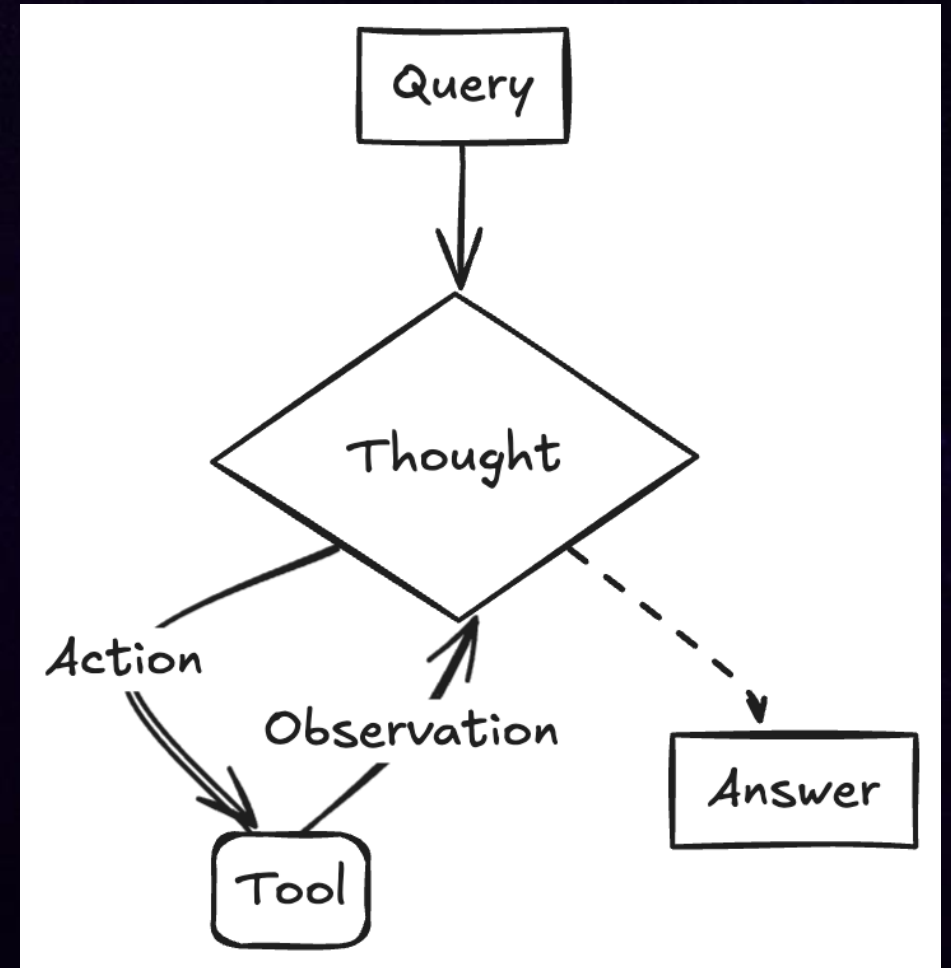
# 2. Launch MPS daemon on all nodes
echo "Starting MPS daemon..."
export TACC_TASKS_PER_NODE=1 # Force 1 task/node
ibrun -np $SLURM_NNODES nvidia-cuda-mps-control -d
unset TACC_TASKS_PER_NODE
sleep 5 # Wait for daemons to initialize

# 3. Run your CUDA application echo "Launching application..."
ibrun ./your_cuda_executable # Replace with your executable
```

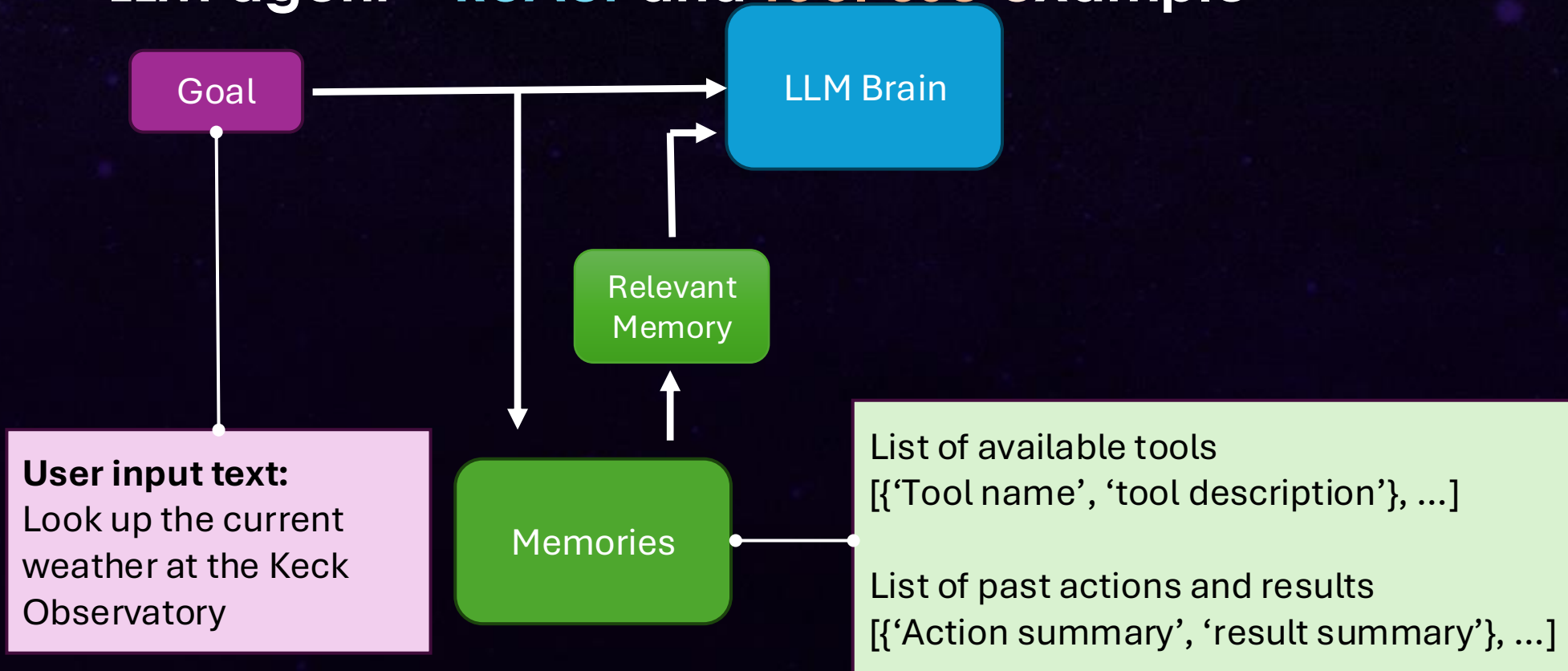
Refer Vista User Guide: <https://docs.tacc.utexas.edu/hpc/vista/>

ReAct

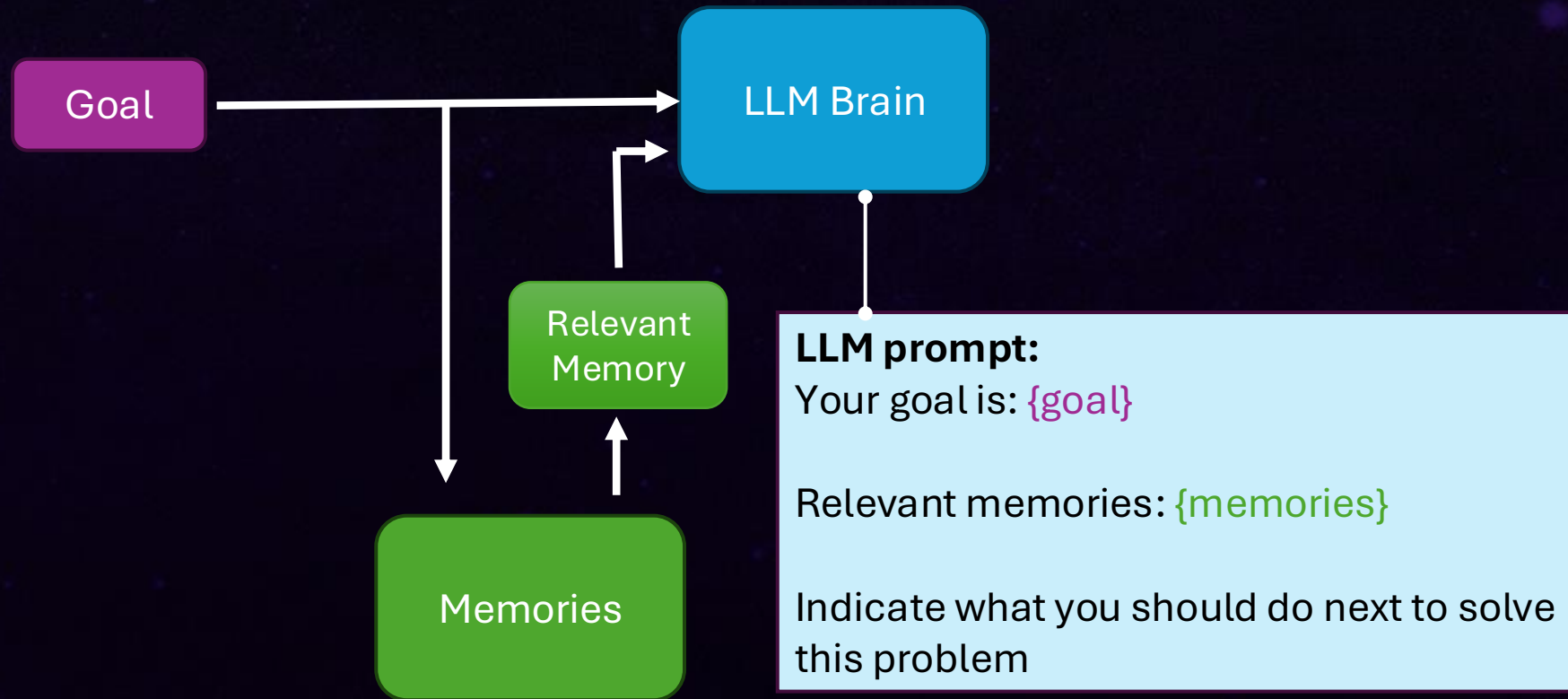
- ▶ ReAct agents are AI systems that reason over their current environments state, then decide what action to perform next
- ▶ They operate in a loop of reasoning, acting, and observing, allowing for dynamic decision-making and real-time adjustments based on feedback.
- ▶ Contrasting ReAct agents with goal decomposition, ReAct agents only think 1 step ahead when reasoning.



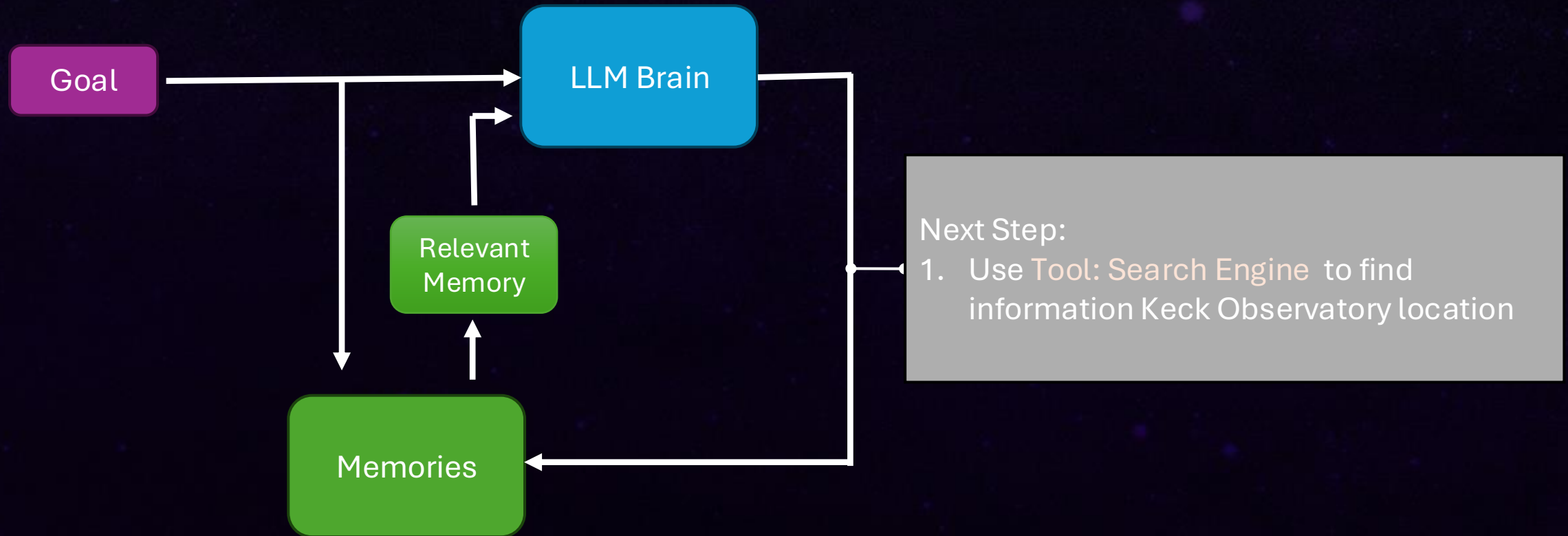
LLM agent – ReAct and tool use example



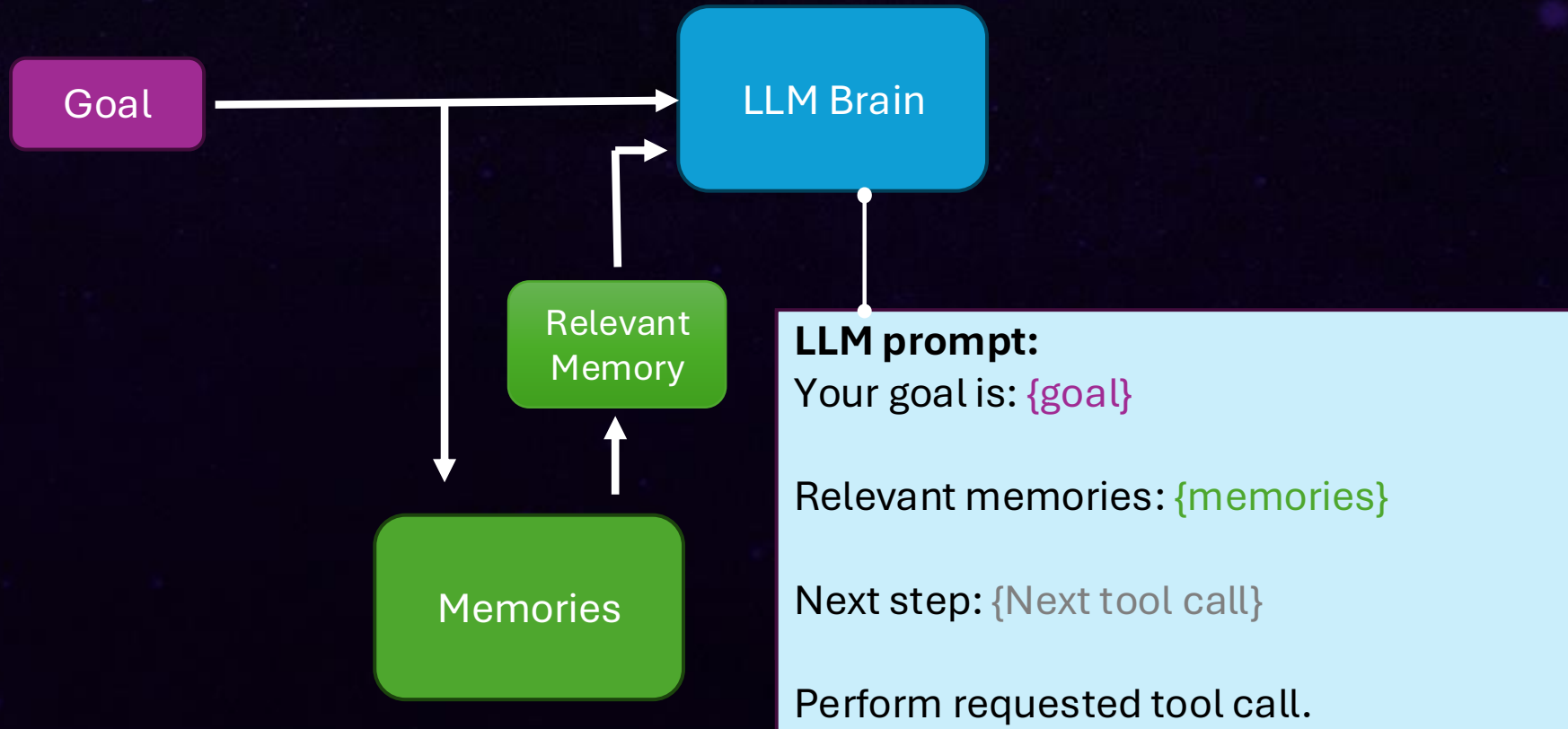
LLM agent – ReAct and tool use example



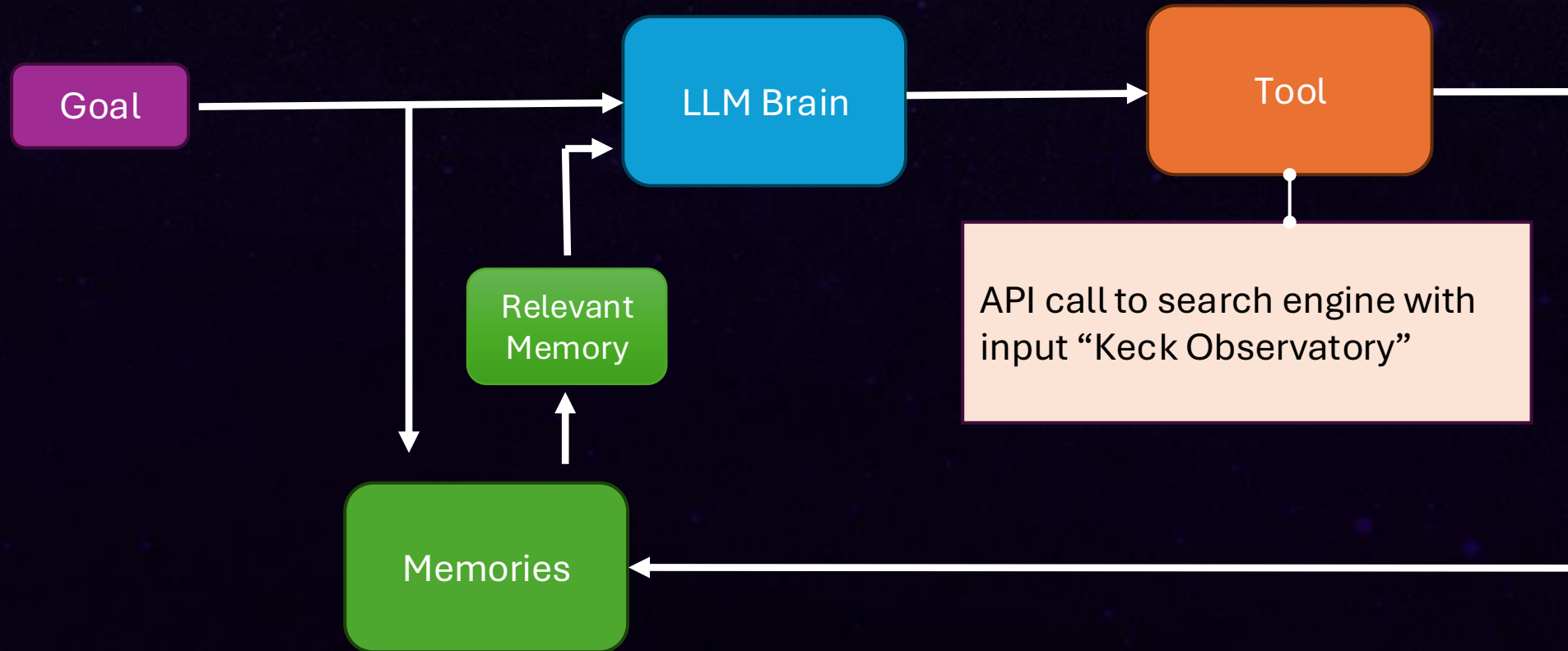
LLM agent – ReAct and tool use example



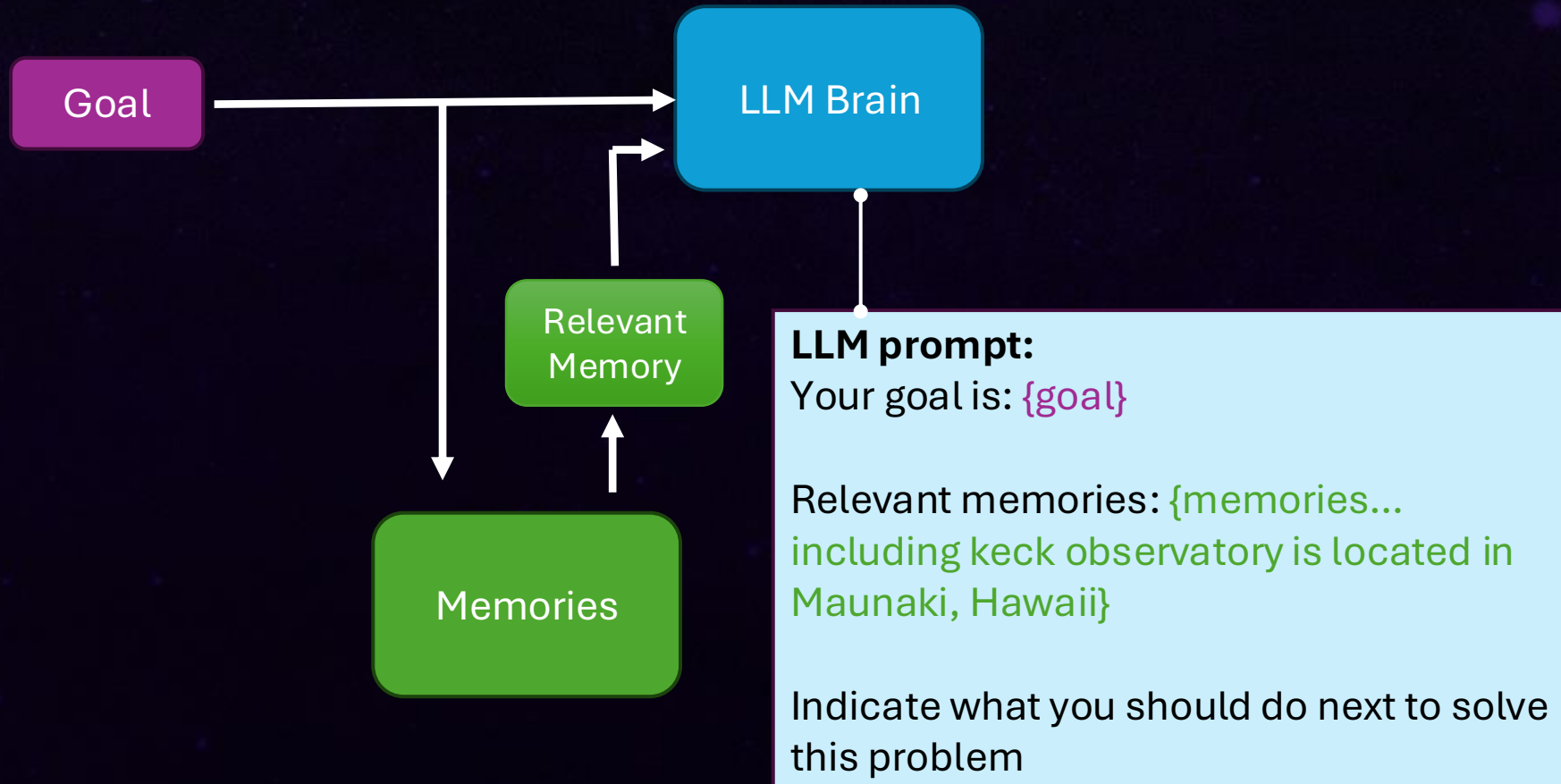
LLM agent – ReAct and tool use example



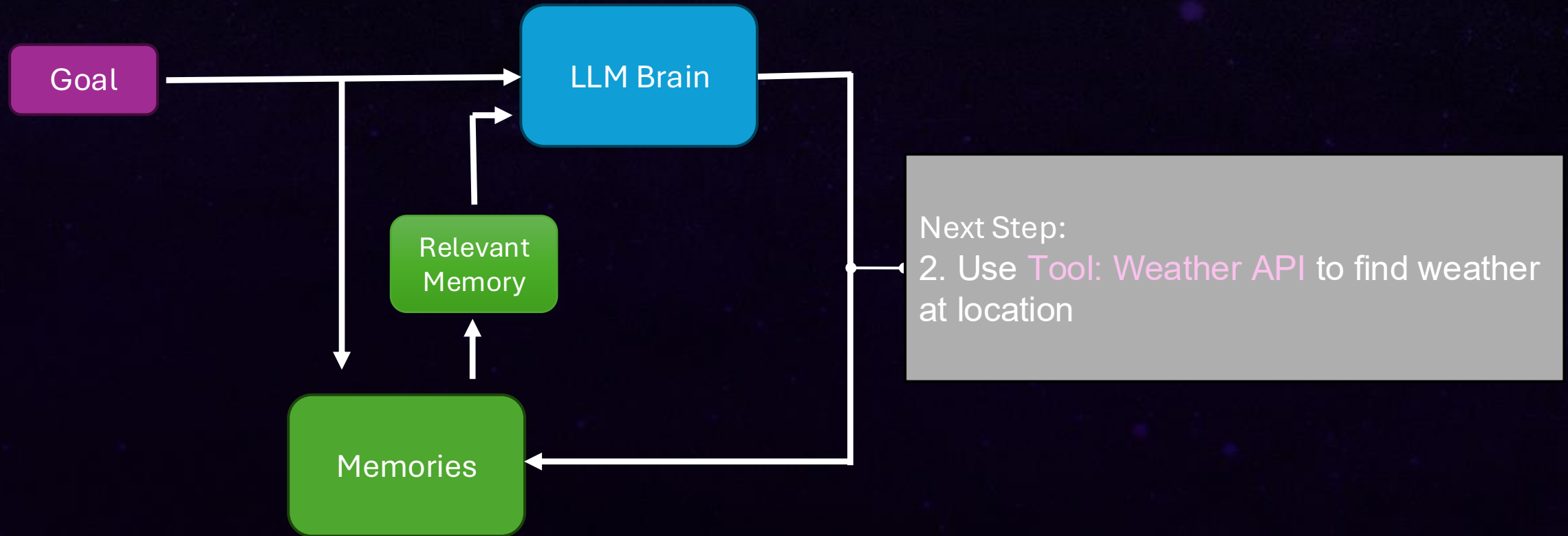
LLM agent – ReAct and tool use example



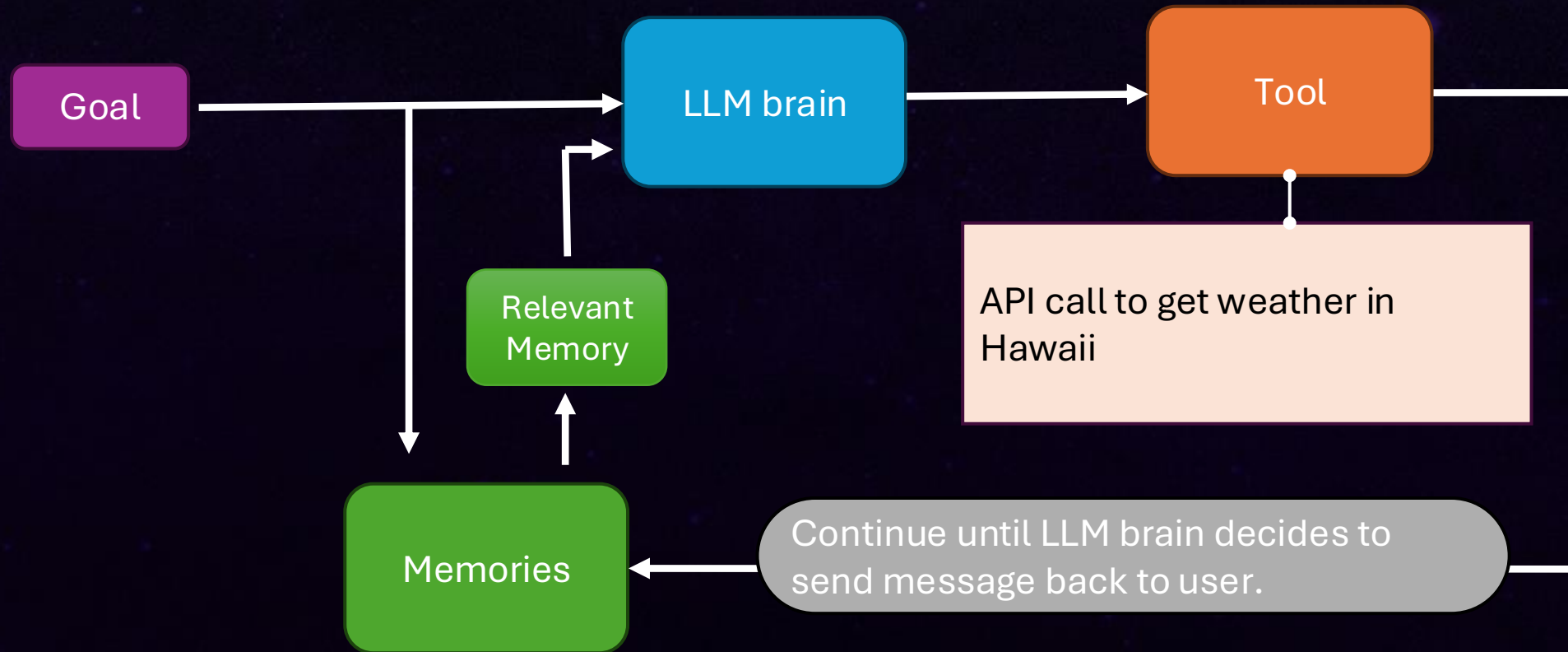
LLM agent – ReAct and tool use example



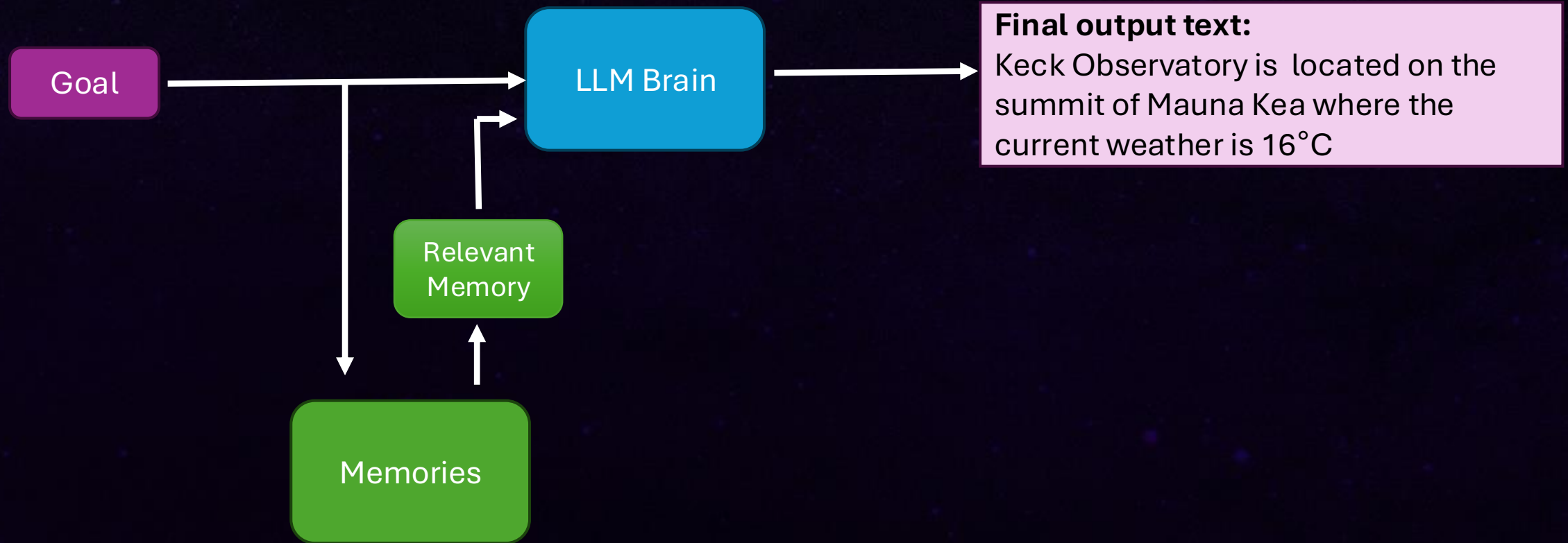
LLM agent – ReAct and tool use example



LLM agent – ReAct and tool use example



LLM agent – ReAct and tool use example



Agents can perform planning and reflection on their memories as a form of learning

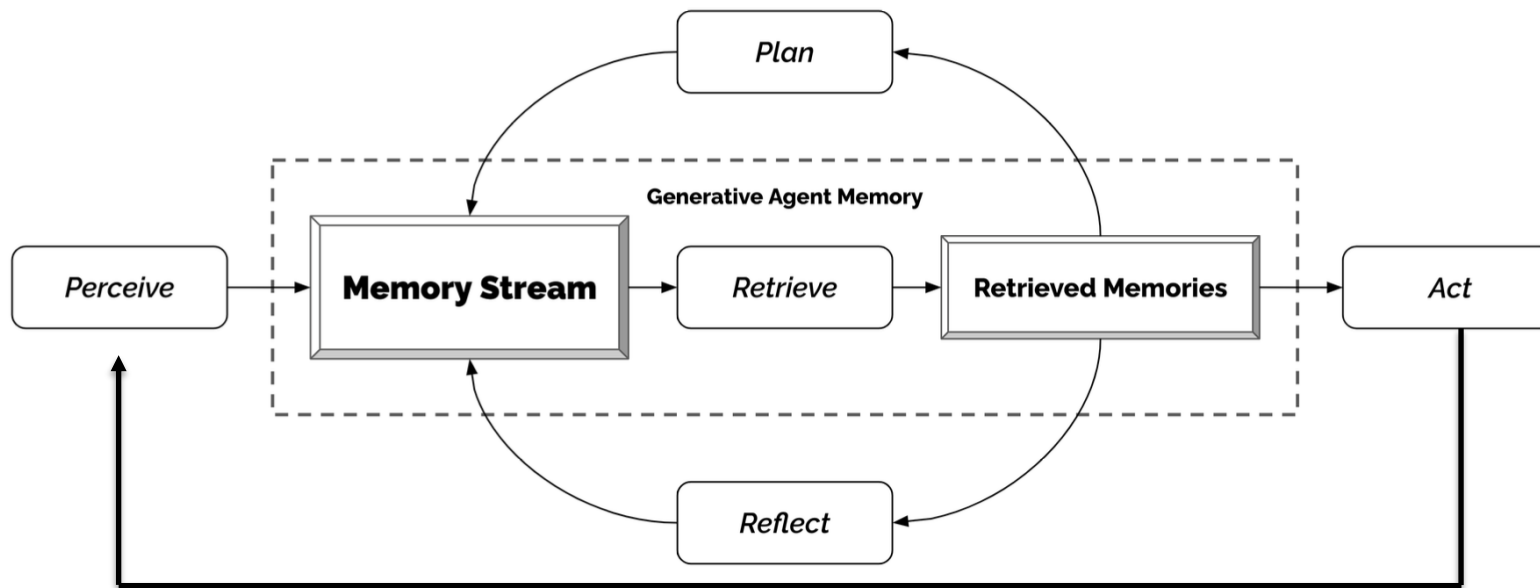
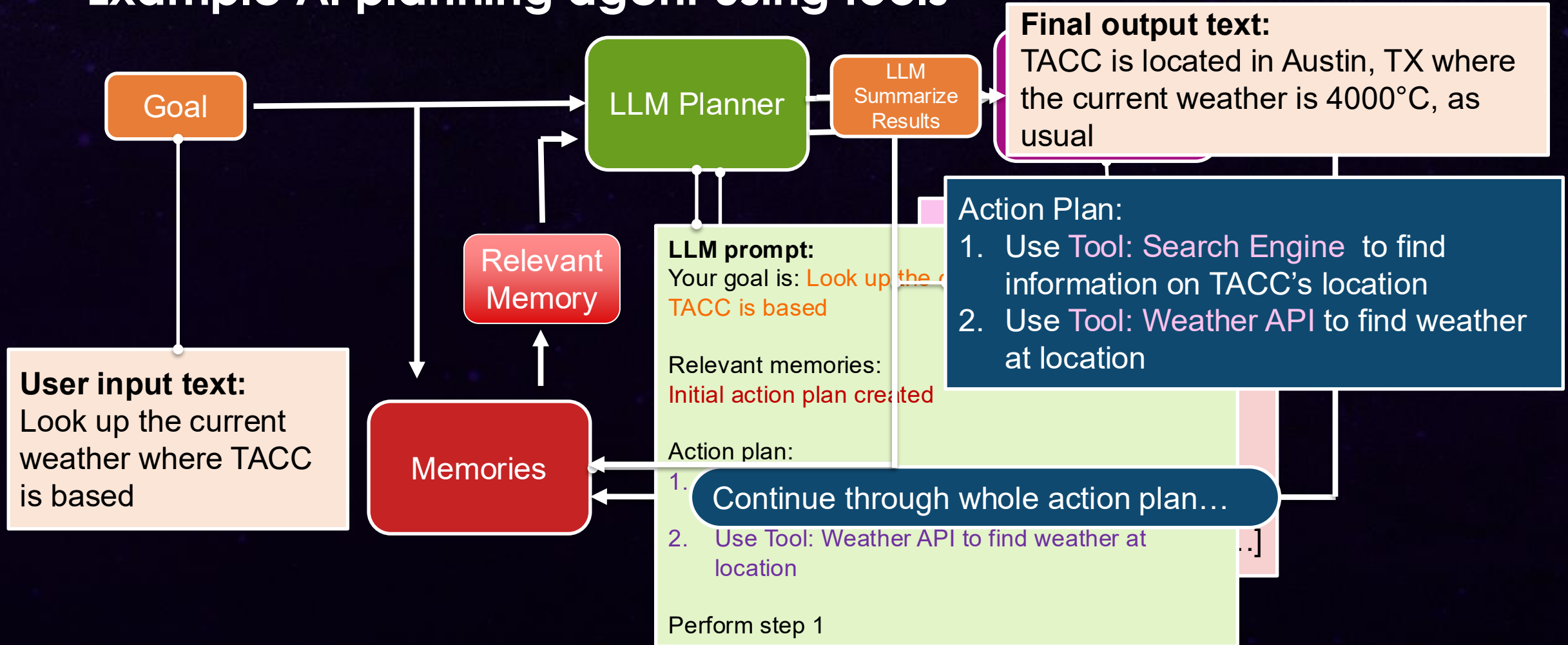


Figure 5: Our generative agent architecture. Agents perceive their environment, and all perceptions are saved in a comprehensive record of the agent's experiences called the memory stream. Based on their perceptions, the architecture retrieves relevant memories and uses those retrieved actions to determine an action. These retrieved memories are also used to form longer-term plans and create higher-level reflections, both of which are entered into the memory stream for future use.

Example AI planning agent using tools



ReAct vs Planning

	ReAct	Planning
Pros	Adaptability to new information	Clear plan generated upfront
	Can be adapted to have an interactive experience	Better at more complex, long range tasks
	Can be more responsive	Better accuracy
Cons	Can get stuck in loops	May not adapt as quick as ReAct agents to changes in environment
	Limited reasoning (one step ahead), depending on task can be suboptimal	
	Prone to cascading errors	May be a less responsive (upfront time spent in long range planning)
Token Consumption	Long Range Tasks: High (one LLM call per reasoning + action pair)	Long Range Tasks: Lower than ReAct (avoids repeated LLM calls by upfront planning)
	Smaller Tasks: Low (less steps, simpler planning)	Smaller Tasks: High (upfront planning consumes more tokens)

TACC_ExAI

TACC Explainable AI Framework

- Agent framework being developed at TACC
 - Led by Gabriel Jaffe
- Memory (Persistent Sessions)
- Interactive (chat), development and experiment modes
- Personality reloading
- Modular Action pipeline
- Has an internal vector store
 - Full conversation history embeddings
- Supports OpenAI API based backends (and Ollama)
- Built in display module
 - Formatted console output

Actions

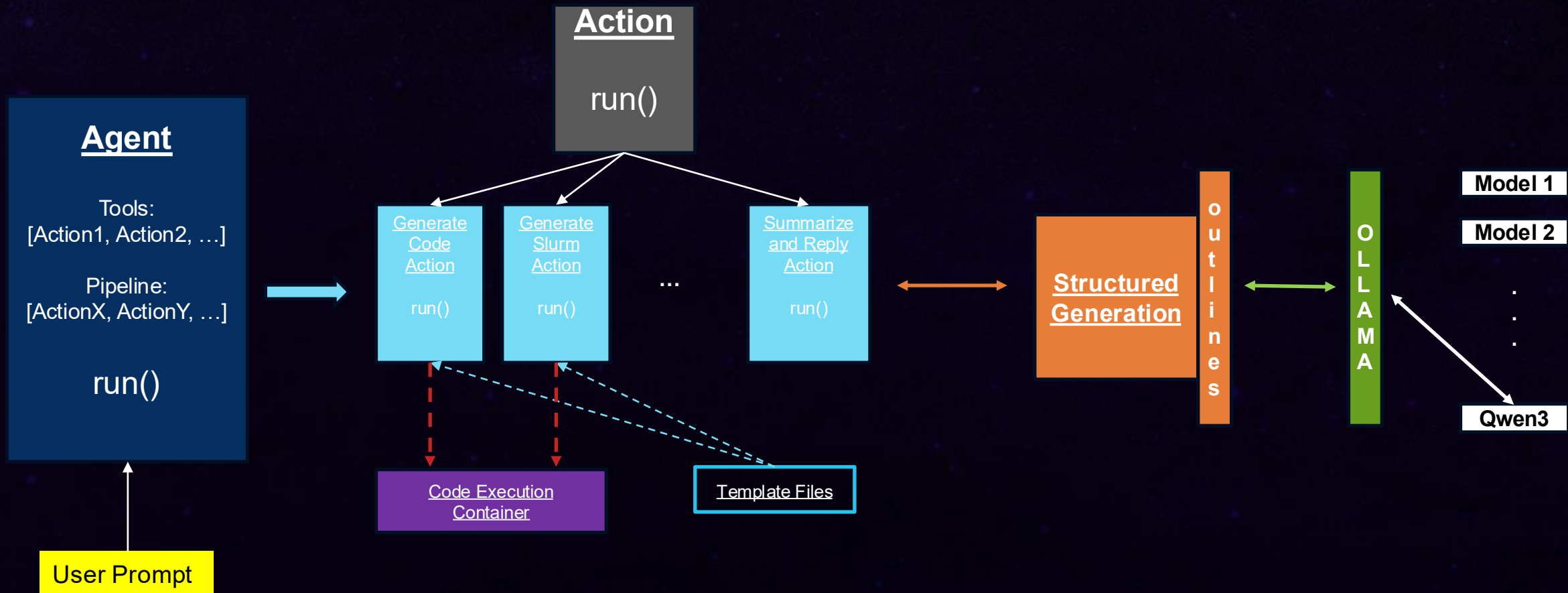
TOOLS

- Summarize and Reply
 - Summarize conversation so far
 - Generate reply to user request
- Generate Code
 - Generate Code
 - Test execution in sandbox container
 - Write out code file
- Generate Slurm Script
 - Load in prompt template
 - Generate Slurm script
 - Load in a slurm script template (output file)
 - Write out the script file

PIPELINE

- Create Plan
 - Formulate prompt
 - User request
 - Available tools
 - Response structure
 - Generate step by step plan
 - Which actions to call
- Run Plan
 - Run each step in generated plan
 - Manages context between each step

Architecture



Notebook walkthrough

Start the TAP Session

Log on to the Analysis Portal with your TACC user credentials

<https://tap.tacc.utexas.edu/>

Welcome to the TACC Analysis Portal

simple access to TACC's analysis resources

Log In to TAP



Start the Session

Launch a Jupyter Notebook

System: Vista

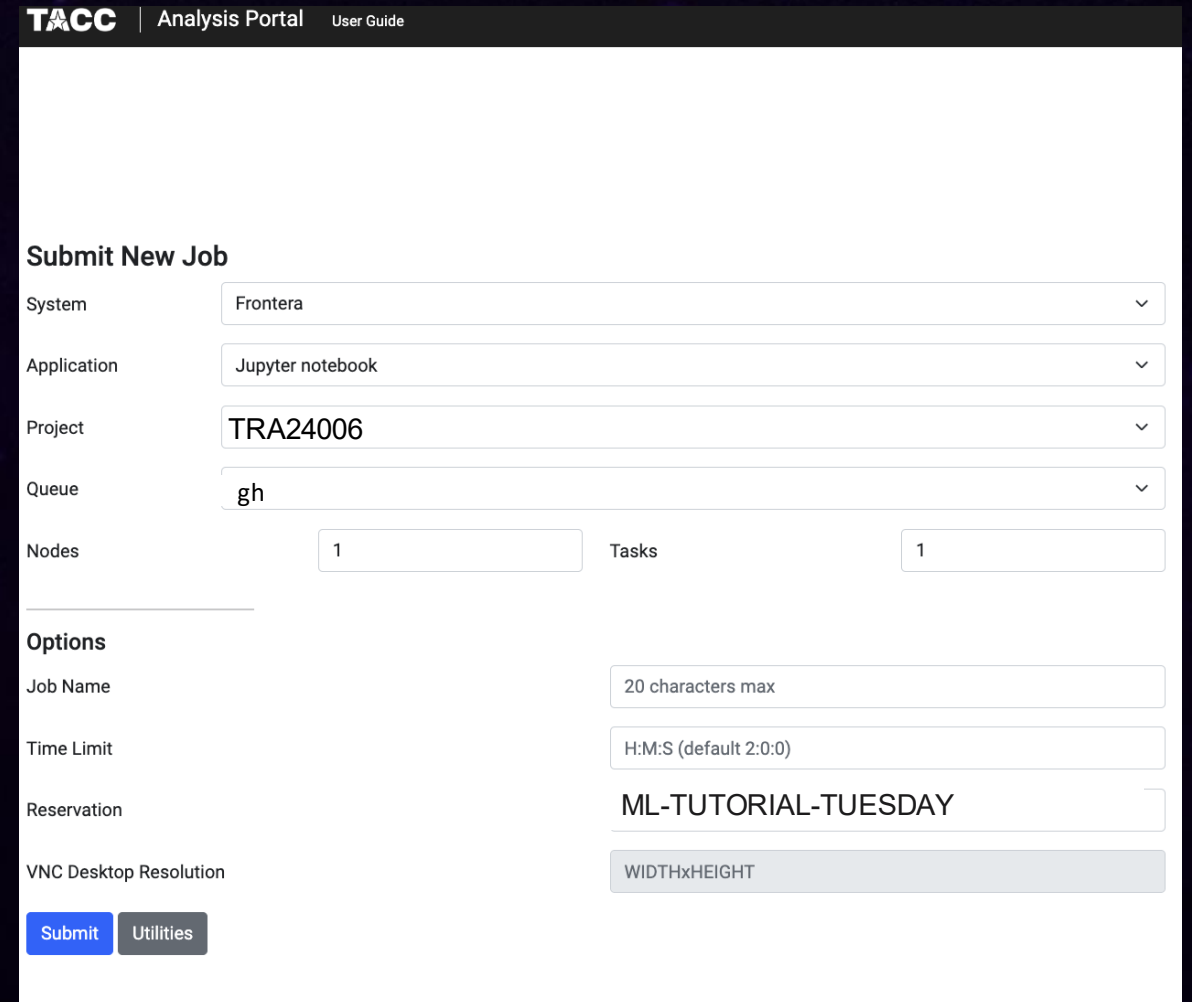
Application: Jupyter notebook

Project: TRA24006

Queue: gh

Time Limit: 02:00:00

Reservation: ML-TUTORIAL-TUESDAY



The screenshot shows the 'Submit New Job' form in the TACC Analysis Portal. The form includes dropdown menus for System (Frontera), Application (Jupyter notebook), and Project (TRA24006). It also has input fields for Queue (gh), Nodes (1), and Tasks (1). Below these are sections for Options, including Job Name (20 characters max), Time Limit (H:M:S, default 2:0:0), Reservation (ML-TUTORIAL-TUESDAY), and VNC Desktop Resolution (WIDTHxHEIGHT). At the bottom are 'Submit' and 'Utilities' buttons.

TACC | Analysis Portal | User Guide

Submit New Job

System: Frontera

Application: Jupyter notebook

Project: TRA24006

Queue: gh

Nodes: 1 Tasks: 1

Options

Job Name: 20 characters max

Time Limit: H:M:S (default 2:0:0)

Reservation: ML-TUTORIAL-TUESDAY

VNC Desktop Resolution: WIDTHxHEIGHT

[Submit](#) [Utilities](#)

Starting Jupyter

TAP Job Status

Job: plzwork

Status: RUNNING

Start: 03/20/2026 12:43

End: 03/20/2026 14:43

Refresh: in 886 seconds

Message:

TAP: Your session is running at https://vista.tacc.utexas.edu:60543/?token=ZAHzf-r_BuJwz4lLEryfAOLdzTkDxo2JgET2pc8S-l8

Connect

End Job

Show Output

Back to Jobs

Thank You

- data@tacc.utexas.edu